Technology Stack Evaluation — Backend AI Engineering Portfolio

Name: Sibgha Mursaleen
Date: 17 July 2026

1. Purpose and Constraints

This document evaluates three technology stack options for building a personal portfolio to showcase four backend-focused projects: a CRUD API (FastAPI, PostgreSQL, Docker), the VNIAS-IJILS Backend, a Complaint Management System (ASP.NET Core), and a full-stack e-commerce project.

The portfolio requires: a Home page, Projects page, individual Project Case Studies, an About page, and a Contact page. It will use screenshots, GitHub repository links, documentation, and written case studies.

Constraints guiding the evaluation:

- Free tools only
- The builder is still learning frontend development
- The stack must be one the builder can confidently maintain, explain, and expand
- No backend is required at launch the portfolio is a static showcase

2. Stack Option 1: Plain HTML, CSS, and JavaScript

Description: Each page (Home, Projects, About, Contact, and one per case study) is hand-written as a separate .html file. A shared CSS file maintains consistent styling across all pages.

Hosting: GitHub Pages (free). The repository is pushed to GitHub, Pages is enabled in the repository settings, and the site is served at a github.io URL.

Backend required: No.

Portfolio fit: Adequate for a small number of static pages. Requires no new tooling, which lowers the barrier to getting something live quickly.

Trade-offs:

- Shared elements (navigation, header, footer) must be copied into every file by hand. A single update to the navigation means editing it in every page.
- No component reuse. With four separate case studies, this becomes repetitive.

- No build step, which means nothing can break during a build — but also no structure to prevent duplication.

3. Stack Option 2: React with Vite (Static)

Description: A component-based build. Shared elements header, footer, project card are built once as reusable components. Case study pages are generated from a single template component that accepts different project data, rather than one file per project.

Hosting: Vercel or Netlify (free tier). The repository connects to the hosting platform, which rebuilds and redeploys automatically on every push to GitHub.

Backend required: No. The site is built into static files at build time and served as static assets.

Portfolio fit: Strong fit for four projects with repeated page structure (each case study needs the same layout: overview, tech stack, screenshots, links, write-up). Solves the duplication problem directly.

Trade-offs:

- Introduces new concepts: components, props, a build step, and package management via npm.
- Requires an additional library for page navigation, since routing is not built into Vite by default.
- One more layer between "code is written" and "site is live" compared to Option 1, meaning more points where something could be misconfigured.

4. Stack Option 3: Next.js

Description: A full React framework with built-in routing, layouts, and server-side rendering. Supports API routes, meaning the same project could later support a working backend without switching frameworks.

Hosting: Vercel (free tier, built by the same team as Next.js).

Backend required: Not at launch. The site can be exported statically. The backend capability exists but is unused unless specifically built.

Portfolio fit: Exceeds what is needed for a five-page static showcase. Its advantages (server-side rendering, API routes) address problems the portfolio does not currently have.

Trade-offs:

- Steepest learning curve of the three: routing conventions, the distinction between Server Components and Client Components, and multiple rendering modes must all be understood to use it correctly.
- Given the constraint of "still learning" and "must be able to confidently explain," this is the option most likely to result in code that works but isn't fully understood by its author a risk if asked to explain design decisions in an interview or review.
- Its main advantage (future backend integration) is speculative rather than a current requirement.

5. Comparison Summary

Criterion	**Plain HTML/CSS/JS**	**React + Vite**	**Next.js**
Learning curve	None new	Moderate	Steep
Reuse across 4 case studies	Manual, repetitive	Built-in via components	Built-in via components
Free hosting	GitHub Pages	Vercel / Netlify	Vercel
Backend required at launch	No	No	No
Room to add backend later	No	Limited	Yes
Can be fully explained by the builder today	Yes	Yes, with focused study	Least likely

6. Decision

Selected stack: React with Vite.

Rejected: Plain HTML/CSS/JavaScript and Next.js.

7. Rationale

I chose React with Vite because it gives me reusable components for the case-study pages, without the added complexity of Next.js features I don't need yet, such as server-side rendering and API routes. Plain HTML/CSS would mean repeating the same layout by hand across five or more pages, which becomes harder to maintain as the portfolio grows. React

with Vite is the balance point between the two extremes: it is organized enough to scale across four project case studies, free to deploy, and simple enough that I can fully explain and maintain every part of it.

Maintainability assessment: This stack is realistic for me to maintain. It introduces a small, well-documented addition react-router-dom, for page navigation but does not require learning rendering modes, server components, or framework-specific routing conventions the way Next.js would. I can explain every decision in this stack because none of it is hidden framework behavior; it is component structure and routing I set up myself.